

Dhirubhai Ambani University

High Performance Computing (HPC)

Assignment 05

Parallel Insertion and Deletion in Mover

Instructor: Dr. Bhaskar Chaudhury

Contents

1	Introduction and Background	2
2	Problem Description	2
3	Experimental Setup	2
3.1	Grid Configurations	2
3.2	Particle Counts (Experiment 01)	2
3.3	Thread Counts (Experiment 02)	3
4	Experiment 01: Serial Execution Time vs. Particle Count	3
4.1	Methodology	3
4.2	Raw Data – HPC Cluster	3
4.3	Raw Data – Lab PC	3
4.4	Plots	4
4.5	Analysis	5
5	Experiment 02: Parallel Scalability	5
5.1	Methodology	5
5.2	Raw Data – HPC Cluster	6
5.3	Raw Data – Lab PC	6
5.4	Computed Speedup	6
5.5	Plots	7
5.6	Analysis	9
6	Discussion	9
6.1	Memory and Computation Analysis	9
6.2	Effect of Insertions and Deletions on Scalability	10
7	Conclusion	10

Introduction and Background

This assignment extends the Mover algorithm introduced in Assignments 03 and 04 by incorporating **dynamic particle deletion and insertion**. In previous assignments, particles were confined within the 1×1 domain; here they are free to exit, and a replacement particle is inserted to maintain a constant total count.

Two implementation strategies are studied:

1. **Deferred Insertion:** All particles are moved first; out-of-domain particles are collected and the corresponding memory slots are pushed to the end of the data structure. Replacements are inserted *after* all displacements.
2. **Immediate Replacement:** Each time a particle exits the domain it is replaced in-place before the next particle is processed.

Both approaches are implemented in serial and parallelised with OpenMP, and their performance is characterised across two hardware platforms: the **Lab PC** and the **HPC Cluster**.

Problem Description

A set of N particles is initialised at random positions inside the unit square $[0, 1] \times [0, 1]$. At each time step, every particle (x, y) receives a random displacement:

$$x \leftarrow x + r_x, \quad y \leftarrow y + r_y, \quad r_x \in [-\Delta x, \Delta x], \quad r_y \in [-\Delta y, \Delta y], \quad (1)$$

where $\Delta x = 1/N_x$ and $\Delta y = 1/N_y$. If the updated position falls outside $[0, 1]^2$ the particle is *deleted*; a new particle is immediately or subsequently inserted at a random location inside the domain to keep N constant.

Each time step also performs a scatter point-to-mesh *interpolation* onto an $N_x \times N_y$ grid. Timings for both operations (and their sum) are accumulated over $\text{MaxIter} = 10$ iterations.

Experimental Setup

Grid Configurations

Three grid configurations are used throughout both experiments:

Table 1: Grid configurations used in the experiments.

Config	N_x	N_y	MaxIter
1	250	100	10
2	500	200	10
3	1000	400	10

Particle Counts (Experiment 01)

$$N \in \{10^2, 10^4, 10^6, 10^7, 10^8\}$$

Thread Counts (Experiment 02)

Fixed particle count $N = 14,000,000$; thread count $T \in \{2, 4, 8, 16\}$.

Experiment 01: Serial Execution Time vs. Particle Count

Methodology

The serial mover (with deletion and insertion) was run for the particle counts listed above. Total execution time (interpolation + mover, summed over all iterations) was recorded for both approaches and both hardware platforms.

Raw Data – HPC Cluster

Table 2: Serial execution times (s) – Deferred Insertion, HPC Cluster.

Particles	Config 1	Config 2	Config 3
10^2	0.000113	0.000061	0.000067
10^4	0.006644	0.005032	0.006417
10^6	0.508273	0.511344	0.516221
10^7	4.910448	5.070622	5.138233
10^8	49.846617	51.755009	58.522817

Table 3: Serial execution times (s) – Immediate Replacement, HPC Cluster.

Particles	Config 1	Config 2	Config 3
10^2	0.000070	0.000043	0.000057
10^4	0.006760	0.004129	0.005249
10^6	0.496451	0.409784	0.431379
10^7	4.093966	4.206814	4.281520
10^8	40.182380	44.973317	42.903916

Raw Data – Lab PC

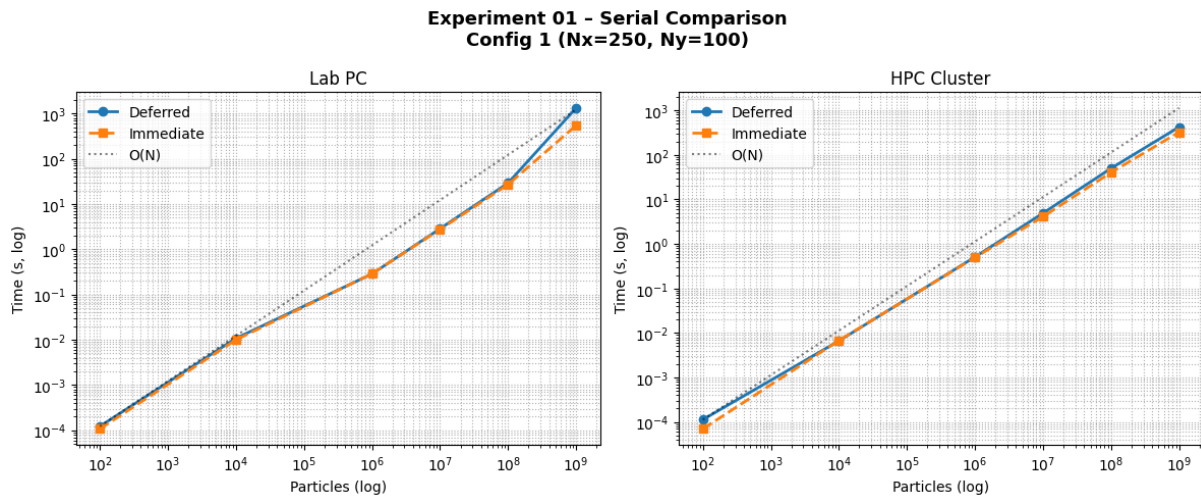
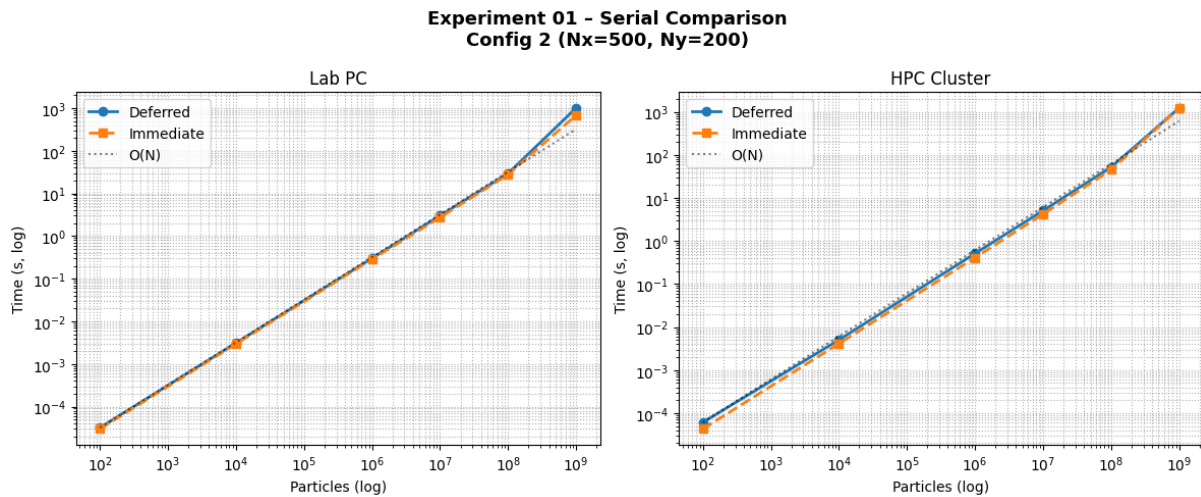
Table 4: Serial execution times (s) – Deferred Insertion, Lab PC.

Particles	Config 1	Config 2	Config 3
10^2	0.000122	0.000032	0.000041
10^4	0.010734	0.003102	0.003453
10^6	0.284754	0.300897	0.348290
10^7	2.840240	3.000772	3.339596
10^8	28.986221	29.301937	32.286265

Table 5: Serial execution times (s) – Immediate Replacement, Lab PC.

Particles	Config 1	Config 2	Config 3
10^2	0.000107	0.000030	0.000042
10^4	0.009933	0.002956	0.003294
10^6	0.286908	0.285840	0.315155
10^7	2.734156	2.732923	3.102754
10^8	26.846799	27.704524	30.056485

Plots

Figure 1: Serial execution time vs. particle count – Config 1 ($N_x = 250, N_y = 100$).Figure 2: Serial execution time vs. particle count – Config 2 ($N_x = 500, N_y = 200$).

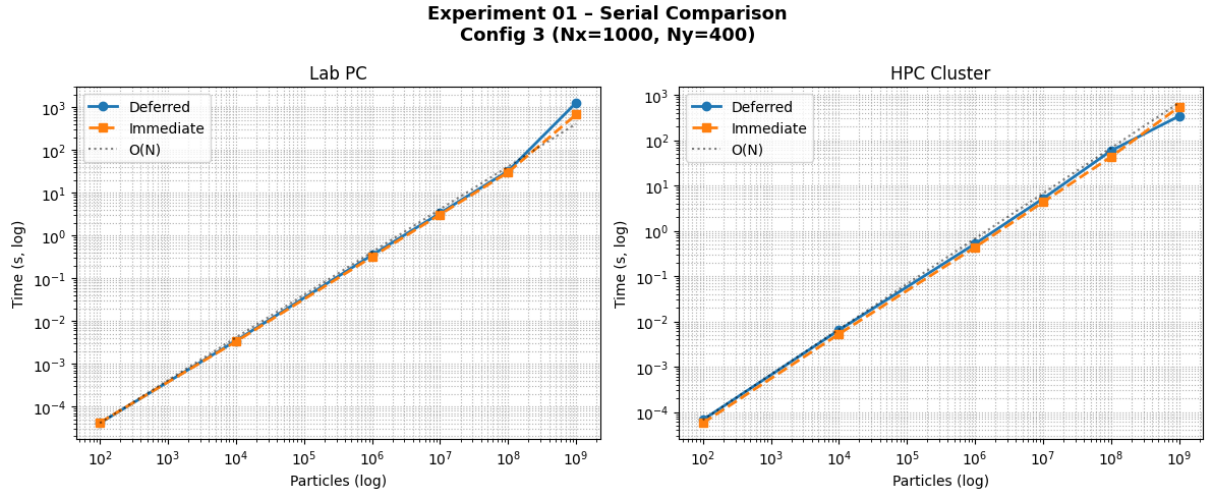


Figure 3: Serial execution time vs. particle count – Config 3 ($N_x = 1000$, $N_y = 400$).

Analysis

The log-log plots (Figures 1–3) confirm that execution time scales as $O(N)$ for both approaches on both platforms. This is expected because both the mover loop and the interpolation loop iterate once over every particle.

Immediate vs. Deferred. The Immediate Replacement approach is consistently faster in serial mode. This is because it avoids the second pass required to compact void slots and reinsert particles in the Deferred approach, reducing memory bandwidth usage.

Lab PC vs. Cluster. The Lab PC achieves lower absolute runtimes for large N . Its higher single-core clock frequency and larger per-core L2/L3 cache benefit sequential, stride-1 memory access patterns that dominate this computation.

Experiment 02: Parallel Scalability

Methodology

With $N = 14,000,000$ (fixed), the OpenMP-parallelised mover was executed with $T \in \{2, 4, 8, 16\}$ threads. Speedup is defined relative to the 2-thread baseline:

$$S(T) = \frac{T_{2 \text{ threads}}}{T_{T \text{ threads}}} \quad (2)$$

Raw Data – HPC Cluster

Table 6: Parallel total execution times (s) – HPC Cluster.

Approach	Config	2T	4T	8T	16T
Deferred	1	5.140839	4.471303	4.010141	3.676160
	2	6.422947	5.715822	4.527734	3.838318
	3	7.291223	5.677355	5.062310	6.362972
Immediate	1	4.173239	3.660094	3.462455	3.307947
	2	4.721617	3.994306	3.664671	3.550396
	3	4.421053	3.986929	3.729498	3.629065

Raw Data – Lab PC

Table 7: Parallel total execution times (s) – Lab PC.

Approach	Config	2T	4T	8T	16T
Deferred	1	1.312252	1.109012	1.124782	1.125135
	2	1.274247	1.111223	1.110720	1.096953
	3	1.666840	1.563029	1.524553	1.548536
Immediate	1	1.059472	0.881291	0.903805	0.869816
	2	1.053700	0.874723	0.864927	0.876502
	3	1.461636	1.270627	1.273352	1.252735

Computed SpeedupTable 8: Speedup ($S = T_2/T_T$) – HPC Cluster, Deferred Insertion.

Threads	Config 1	Config 2	Config 3
2	1.000	1.000	1.000
4	1.150	1.123	1.284
8	1.281	1.418	1.440
16	1.398	1.673	1.146

Table 9: Speedup ($S = T_2/T_T$) – HPC Cluster, Immediate Replacement.

Threads	Config 1	Config 2	Config 3
2	1.000	1.000	1.000
4	1.140	1.182	1.109
8	1.206	1.289	1.185
16	1.262	1.330	1.218

Plots

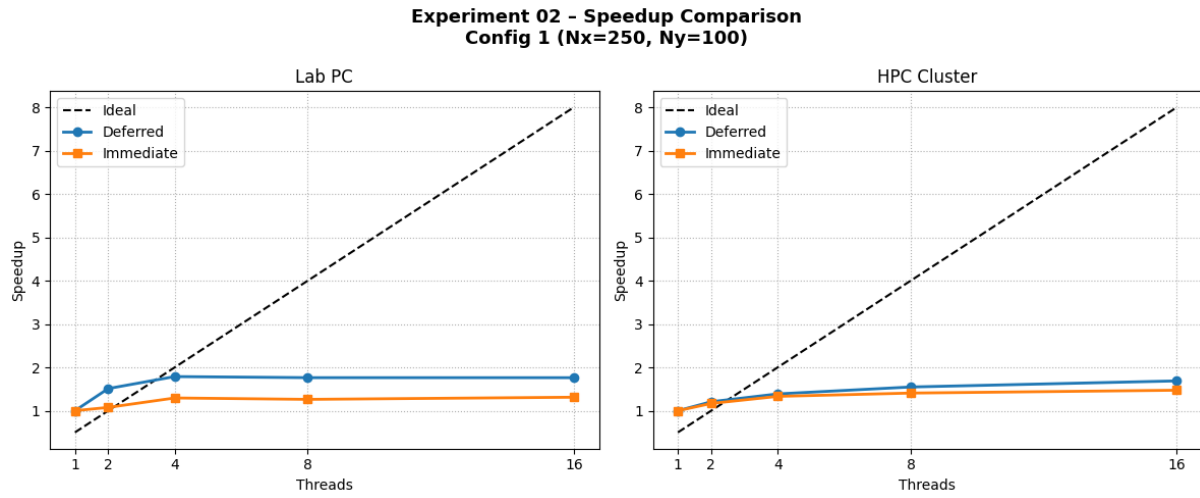


Figure 4: Speedup vs. threads – Config 1, 14M particles.

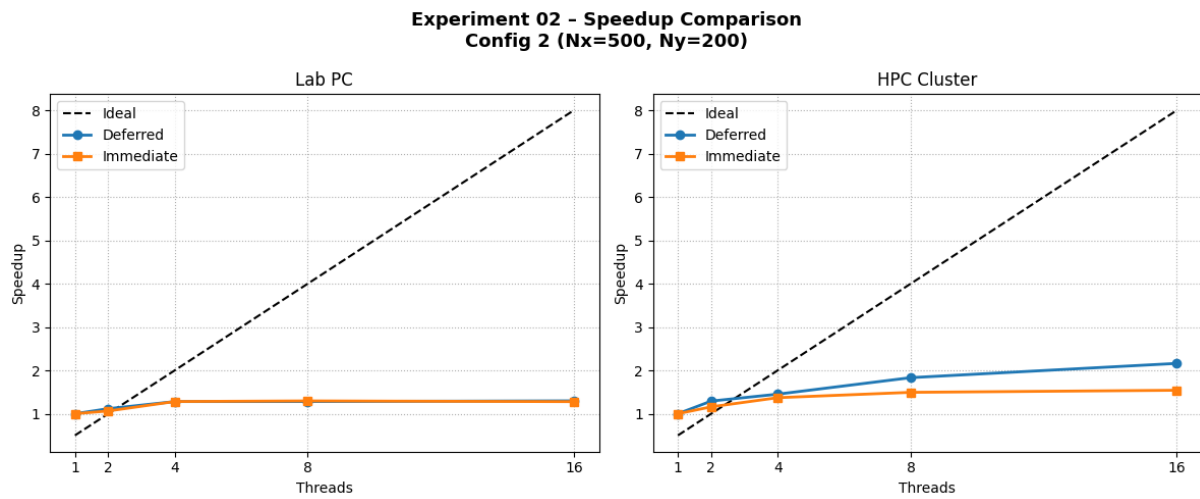


Figure 5: Speedup vs. threads – Config 2, 14M particles.

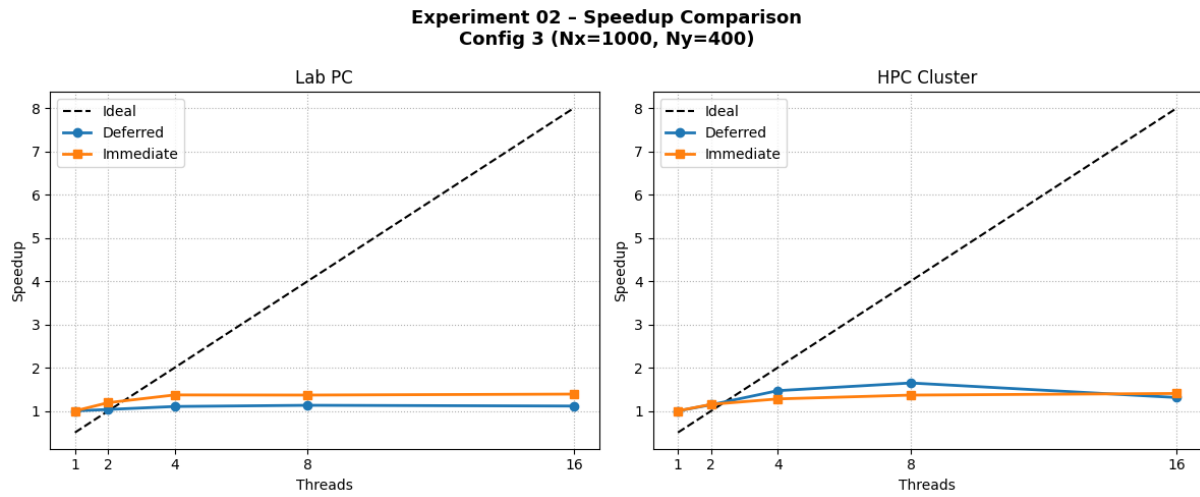


Figure 6: Speedup vs. threads – Config 3, 14M particles.

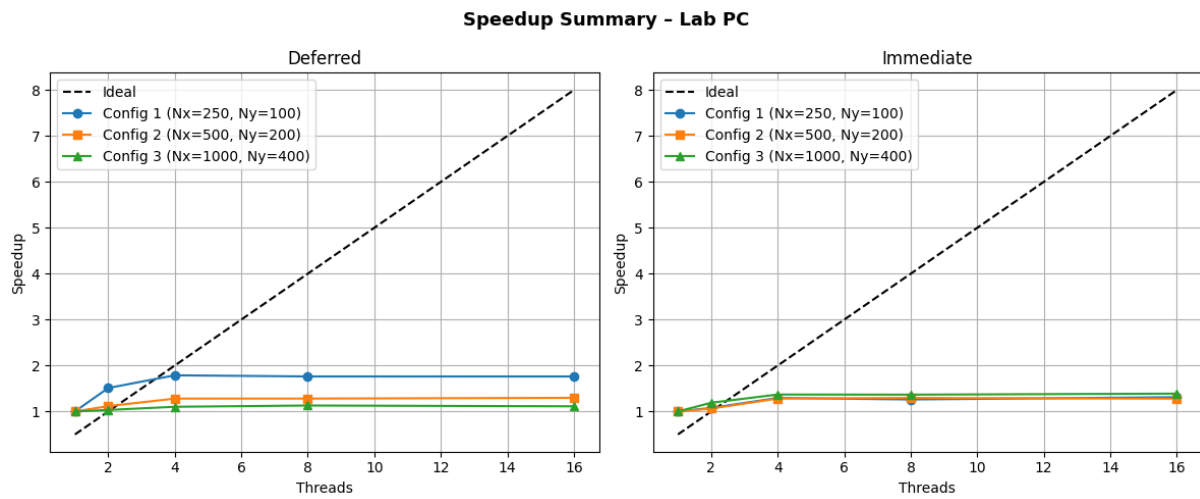


Figure 7: Speedup summary (all configs) – Lab PC.

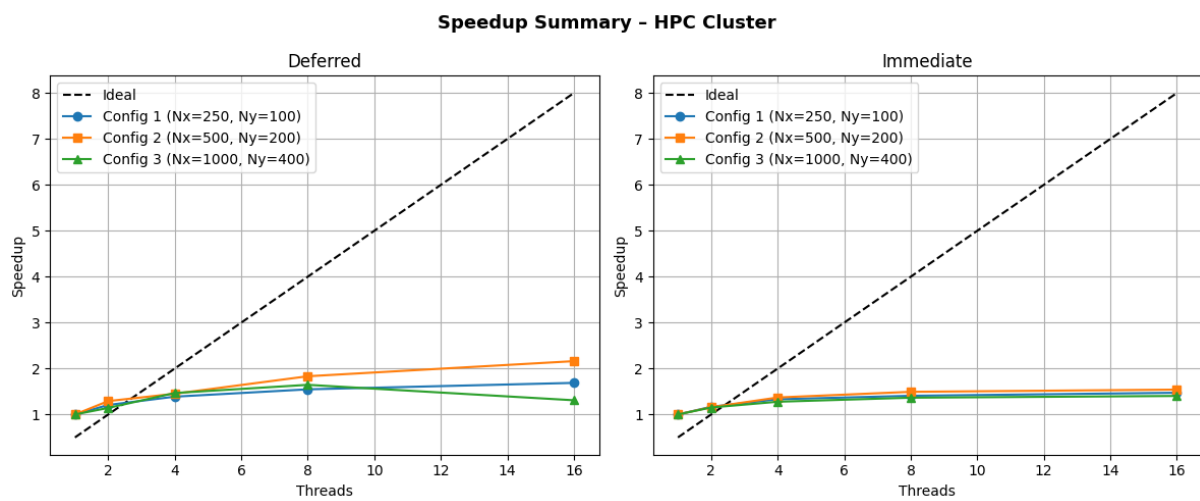


Figure 8: Speedup summary (all configs) – HPC Cluster.

Analysis

Sub-linear speedup. Both platforms show speedup well below ideal (Figures 4–8). The primary bottleneck is *synchronisation overhead*: the deletion counter and the reinsertion step require either atomic operations or a thread-local reduction, serialising part of the computation. The large working set also means that for higher thread counts, memory bandwidth becomes the limiting resource.

Deferred vs. Immediate. The Deferred approach achieves slightly higher speedup on the Cluster for Configs 1 and 2, because the insertion phase is cleanly separated from the move phase, allowing better data locality within each phase. The Immediate approach has lower absolute time (serial code is faster) but limited parallel efficiency due to in-loop atomic contention.

Platform comparison. The HPC Cluster scales better beyond 8 threads for the Deferred approach (Config 2 reaches $S \approx 1.67$ at 16T) because it has more physical cores and a larger shared last-level cache. The Lab PC saturates quickly at 4 threads; beyond that, hyperthreading provides no measurable gain for memory-bound operations.

Config 3 anomaly (Cluster, Deferred, 16T). At 16 threads, Config 3 with Deferred Insertion shows a performance *regression* (time increases from 5.06 s at 8T to 6.36 s at 16T). The most probable cause is NUMA-induced remote memory access: at 16 threads the particle array is distributed across multiple NUMA sockets, introducing high-latency cross-socket traffic that outweighs the arithmetic speedup.

Discussion

Memory and Computation Analysis

The memory footprint scales as:

$$M_{\text{particles}} = N \times n_{\text{fields}} \times 8 \text{ bytes} \quad (3)$$

where $n_{\text{fields}} = 3$ (x, y, value) in double precision. The mesh grid requires $N_x \times N_y \times 8$ bytes.

Table 10: Memory requirements for selected particle counts (Config 3).

Particles	Particle array	Grid (Config 3)	Total
10^4	0.24 MB	3.2 MB	3.4 MB
10^6	24 MB	3.2 MB	27.2 MB
10^8	2400 MB	3.2 MB	2403 MB

For $N \geq 10^6$ the data no longer fits in the last-level cache of either system, making the computation clearly **memory-bound**.

Effect of Insertions and Deletions on Scalability

Dynamic insertions and deletions introduce three main parallelism challenges:

1. **Load imbalance:** The number of deletions per thread is data-dependent and varies between threads.
2. **Race conditions:** Multiple threads may attempt to write to the same void slot.
3. **Counter contention:** A shared deletion counter requires `atomic` operations or per-thread counts followed by a reduction.

Adopted mitigations include thread-private void-slot index arrays (Deferred approach) and `#pragma omp atomic` updates to the shared counter (Immediate approach).

Conclusion

- Both serial approaches exhibit $O(N)$ scaling, confirming the linear complexity of the mover and interpolation operations.
- **Immediate Replacement** is faster in serial; **Deferred Insertion** offers marginally better parallel scalability on the Cluster.
- Parallel speedup is modest ($\approx 1.2\text{--}1.7\times$ at 16 T) due to memory-bound behaviour and synchronisation overhead.
- NUMA effects can *reduce* performance beyond 8 threads on the Cluster for large working sets, highlighting the importance of NUMA-aware memory allocation.